

CC3001 Algoritmos y Estructuras de Datos

Control 1

Prof. Benjamin Bustos (Sec. 1) y Nelson Baloian (Sec. 2)

Fecha: 23 de septiembre de 2010

Tiempo: 1 hora 45 minutos. Una hoja de oficio con apuntes.

Por favor póngale nombre a todas la hojas.

Pregunta 1

Considere la siguiente ecuación de recurrencia:

$$T(n) = 5T(n-1) - 6T(n-2), T(0) = 0, T(1) = 1$$

- i. (1 pto.) Escriba en Java un método recursivo que calcule $T(n)$ para un n dado.

```
int computeT(int n)
```

- ii. (2 ptos.) Analice cuántos llamados recursivos debe realizar el método `computeT` para calcular $T(n)$. Para esto, defina la ecuación de recurrencia correspondiente al método y luego resuélvala.
- iii. (2 ptos.) Resuelva la ecuación de recurrencia de $T(n)$ en forma exacta.
- iv. (1 pto.) Utilice la solución encontrada en la parte (iii) y escriba un método iterativo que calcule $T(n)$. ¿Qué complejidad tiene este algoritmo?

Pregunta 2

Diseñe un algoritmo y escriba el método

```
boolean correcto(char[] x)
```

que retorna *true* si en el arreglo x viene correctamente escrita una lista de números enteros con el siguiente formato: $(n_1, n_2, n_3, \dots, n_k)$. La cantidad de números en la lista es variable. Son válidas las listas “()” (lista vacía) y “(n_1)” (lista con un sólo número). Un número entero se define como una secuencia de dígitos. No deben venir espacios entre medio.

Resuelva este problema en los dos pasos siguientes:

- i. Dibuje el diagrama de estados correspondiente a su algoritmo.
- ii. Escriba el código de su programa.

Pregunta 3

Suponga que se dispone de la siguiente implementación de Nodo de una lista enlazada:

```
class Nodo{
    int valor;
    Nodo sgte;
}
```

Diseñe un algoritmo iterativo y escriba el método

```
Nodo mezclarListasOrdenadas(Nodo Lista1, Nodo Lista2)
```

que recibe como parámetros dos listas *ordenadas* (ascendentemente por *valor*), y retorna una lista correspondiente a la mezcla ordenada de ambas listas. Defina claramente cuáles son los invariantes del problema, la inicialización, la condición de término, en qué parte del código se rompe el invariante, dónde se recupera, etc.